# Revisão da arquitetura do banco: plano de correções (feedback QI Tech)

## Contexto

Na apresentação da prévia da RFC, a QI Tech apontou que:

1. o banco está complexo demais;
2. os diagramas ER não batem com as tabelas reais;
3. existem colunas redundantes (começando por `suitability_question`);
4. `transaction` tem três colunas de conta (`requester_account_id`, `origin_account_id`, `destination_account_id`);
5. a idempotência usa um `request_hash` que não deveria existir.

Este documento traz o diagnóstico completo e as correções, em fases pequenas. Ao fim de cada fase rodamos `pytest -v` e paramos para revisar.

Decisões já tomadas:

- `transaction`, `ledger_entry`, `investment_order` e `position` passam a ter **um dono só: a conta**.
- A `Idempotency-Key` passa a ser **UNIQUE de verdade** (no banco inteiro), sem `request_hash`.
- A tarifa passa a **entrar numa conta do banco** (partidas dobradas na tarifa).
- Enumeradores viram **tipo ENUM nativo do Postgres** (`CREATE TYPE ... AS ENUM`), no banco inteiro.

---

## Respostas às perguntas da banca

### Como `transaction`, `ledger_entry` e `investment_order` se organizam

**Hoje** os donos estão misturados:

| Tabela | Papel | Liga em |
|---|---|---|
| `ledger_entry` | Uma linha do extrato: o efeito no saldo de **uma** conta | conta |
| `transaction` | O pedido que mexe em dinheiro | conta (3 colunas) |
| `investment_order` | O pedido de investimento: produto, cotas, preço | cliente **e** conta **e** transação |
| `position` | Quanto se tem de cada produto | cliente |

A ordem e a posição ligam no cliente para que os investimentos "sobrevivessem" a uma troca de conta. Mas essa situação não existe: a conta só pode ser encerrada com as posições zeradas. E `investment_order` é 1:1 com `transaction` e repete o que ela já tem (valor, conta, chave, hash, e o lado, que o tipo `INVESTMENT_APPLY` / `INVESTMENT_REDEEM` já diz).

**Depois:** tudo pendurado na conta, cada tabela com um papel só.

```
customer
   └── account                      dono de tudo que envolve dinheiro
         ├── transaction            O PEDIDO: tipo, valor, chave de idempotência
         │     ├── ledger_entry     O EXTRATO: 1 linha por efeito em um saldo
         │     └── investment_order O ANEXO do pedido de investimento:
         │                          produto, cotas, preço, custo, vencimento
         └── position               o que a conta tem de cada produto
```

- `transaction` é o **pedido** ("aplique R$ 100"). Todo pedido de dinheiro, de qualquer tipo, é uma linha aqui, e é aqui que mora a chave de idempotência.

- `ledger_entry` são as **linhas do extrato** que esse pedido gerou (a transferência gera várias).
- `investment_order` é o **anexo** que só o pedido de investimento tem. Não tem dono próprio: o dono vem da transação. Fica só com `transaction_id` (único), `product_id`, `quantity`, `unit_price`, `cost_amount`, `redeem_all`, `maturity_date`, `settled_order_id`.
- `position` passa a ser da **conta** (`account_id` + `product_id`).

As rotas continuam `/customer/{ck}/...`: o controller acha a conta do cliente e filtra por ela. A API continua devolvendo `side`, `amount` e `source` na ordem, calculados a partir da transação.

### Por que as três colunas de conta em `transaction` estão erradas

`ledger_entry` já diz exatamente quais contas foram afetadas. Na `transaction` (o *pedido*), `requester` sempre repete `origin` ou `destination`, e no depósito a "origem" nem existe. Fica:

- `account_id NOT NULL`: a conta que fez o pedido (dona da transação);
- `destination_account_id`: só na transferência (`CHECK` amarrando ao tipo `TRANSFER`).

O comprovante e a contraparte do extrato saem disso e do tipo; a resposta da API não muda.

### Como a chave de idempotência é gerada (e o que está errado hoje)

**Nosso código não gera a chave.** Quem chama a API (o app, os testes) gera um UUID novo por operação e manda no cabeçalho `Idempotency-Key`. Só quem chama sabe se está *repetindo* uma tentativa: se a internet cai antes da resposta, o app reenvia o mesmo pedido com a mesma chave, e o banco devolve a resposta da primeira vez sem mexer em dinheiro de novo. Se o servidor gerasse a chave, o reenvio ganharia uma chave nova e o cliente pagaria duas vezes.

**O que está errado hoje:** a chave é única *por dono*, em dois lugares: `UNIQUE(requester_account_id, idempotency_key)` em `transaction` e `UNIQUE(customer_id, idempotency_key)` em `investment_order`. Por isso existe o `request_hash`.

**Bug encontrado lendo o código** (o primeiro teste da Fase 1 reproduz antes de corrigir):

1. o cliente faz um depósito com a chave K: grava `transaction` (conta A, K);
2. depois faz uma aplicação com a mesma chave K;
3. o controller procura uma *ordem* com K, não acha, e trata como pedido novo;
4. ao gravar a `transaction` da aplicação com (conta A, K), o banco recusa (já existe);
5. ninguém trata o erro: a API devolve **500** em vez de 409.

**Depois:**

1. Lê o cabeçalho. Faltou ou não é UUID: 400 (`PPP000003`).

2. Trava as contas (como hoje).

3. Procura a chave em `transaction.idempotency_key` (`UUID UNIQUE`, um lugar só):

   - não achou: pedido novo, executa e grava a chave junto com a transação;
   - achou, mesma conta **e pedido idêntico ao gravado**: repetição, devolve a resposta original (200);
   - qualquer outro caso (outra conta, outro tipo, outro valor, outro destino, outro produto): 409 (`PPP000004`), sem mostrar nada do pedido original (R8).

   "Idêntico" é comparado **com as colunas já gravadas**, sem hash: na transação, `type`, `amount`, destino e `description`; na ordem, lado, produto e `amount`. O resgate total não traz valor, então a ordem grava `redeem_all` (o que o cliente pediu) para essa comparação.

4. Aplicação e resgate usam a chave da própria transação, então as duas rotas se enxergam.

5. A liquidação do vencimento é feita pelo banco, sem pedido de cliente: a transação fica **sem chave** (nula). O `UNIQUE(settled_order_id)` continua impedindo pagar duas vezes. Sai o `uuid5`.

### Como reportamos o capital do cliente

**Hoje** o capital aparece em dois pedaços que ninguém junta:

- `GET /customer/{ck}/account/{ak}` devolve `balance`: só o dinheiro em conta (caixa);
- `GET /customer/{ck}/positions` devolve `total.current_amount`: os investimentos ao preço do dia.

Ninguém responde "quanto o cliente tem no banco" nem "quanto disso está em caixa e quanto em investimentos".

**Descartado: guardar o total numa coluna** (`balance` = patrimônio, mais duas colunas de percentual). O valor dos investimentos muda todo dia com o preço, então a coluna ficaria velha a cada fechamento e o fechamento teria que reescrever todas as contas. Além disso, `balance` hoje é exatamente a soma do extrato (créditos − débitos) e é a linha travada com `FOR UPDATE` para decidir saque e aplicação. Misturar investimento nele quebra esse invariante.

**Escolhido: calcular na leitura**, na rota de posições, que já busca o preço do dia de cada posição. Custo: ler a conta não encerrada do cliente e fazer uma soma. Sem coluna nova, sem escrita. O `total` da resposta ganha:

| Campo | Significado |
|---|---|
| `cash_amount` | `account.balance` da conta não encerrada (0 se não houver) |
| `current_amount` | (já existe) investimentos ao preço do dia |
| `net_worth_amount` | `cash_amount + current_amount`: o capital total no banco |
| `cash_bps` | parte em caixa, em pontos-base (`10000` = 100%) |
| `investments_bps` | `10000 − cash_bps`, para a soma fechar sempre em 100% |

Percentual em pontos-base inteiros, como o `return_bps` que já existe (sem `float`, R6). Patrimônio 0 → os dois percentuais são 0.

### A movimentação do dinheiro está correta?

O núcleo está certo: saldo em centavos `BIGINT` com `CHECK >= 0`, `SELECT ... FOR UPDATE` em ordem de id, todo movimento vira `ledger_entry` com `balance_after`, um único commit. Dois furos:

1. **A tarifa some.** Sai da conta do cliente e não entra em lugar nenhum. Correção: uma conta do banco recebe o crédito `FEE`. A conservação do dinheiro fica mais simples: Σ saldos (com o banco) = depósitos − saques − aplicações + resgates.
2. **O bug de idempotência acima** (500 em vez de 409).

Custo aceito (vai para a RFC): toda transferência trava a conta do banco, então as transferências passam a ser serializadas nela. Se isso virar gargalo, a saída é acumular a tarifa e creditar no fechamento do dia.

---

## Diagnóstico completo de redundâncias

| Tabela | Coluna(s) | Problema | Correção |
|---|---|---|---|
| `transaction` | `requester/origin/destination` | Ver acima | `account_id` + `destination_account_id` |
| `transaction` e `investment_order` | `idempotency_key` + `request_hash` duplicados | Ver acima | Chave só em `transaction`, `UUID UNIQUE`; sem hash |
| `investment_order` | `customer_id`, `account_id` | O dono vem da transação | Tirar |
| `investment_order` | `amount`, `side` | Repetem `transaction.amount` e o tipo da transação | Tirar; o DTO lê da transação |

| Tabela | Coluna(s) | Problema | Correção |
|---|---|---|---|
| `investment_order` | `source` | `MATURITY` ⇔ `settled_order_id` não nulo | Tirar; o DTO calcula (o campo `source` continua na API) |
| `investment_order` | `status` | Só aceita `EXECUTED` | Tirar |
| `position` | `customer_id` | A conta só encerra com posição zerada; dono passa a ser a conta | Trocar por `account_id` |
| `ledger_entry` | `entry_key` | Nunca sai na API (R5 só exige key no que sai) | Tirar |
| `suitability_question` | `code` ('Q1') e `position` (1) | Mesma informação duas vezes | Tirar `position`; ordenar por `code` |
| `suitability_option` | `code` e `points` | Sempre iguais (o seed faz `points = code`). E "os pontos nunca saem na API" é falso: o `code` que sai *é* o ponto | Tirar `points`; pontuação = `code` (documentar) |
| `suitability_answer` | `question_id` | Sai de `option_id` (a opção pertence a uma pergunta) | Tirar; o controller já valida uma resposta por pergunta |
| `suitability_assessment` | `valid_until` | É `created_at + 24 meses` | Tirar; calcular no controller/DTO |
| `suitability_assessment` | `forced_by_rule` | É "perfil gravado ≠ perfil da faixa do `score`" | Tirar; calcular no DTO |
| `model_portfolio` | tabela inteira | 1:1 com `investor_profile` | Juntar: `name` vai para `investor_profile`; `model_portfolio_item.profile_id` |
| `product` | `own_issue` | ⇔ `external_code IS NULL` | Tirar; derivar (ajustar fechamento do dia e RN-REC-06) |
| `account_status`, `transaction_type`, `product_type` | tabelas | Enumerador em tabela + `status_id <> 3` "mágico", enquanto `side` / `direction` / `indexer` são texto | Tipos ENUM nativos |
| `direction`, `category`, `actor`, `indexer`, `liquidity`, `reference_rate.code` | `VARCHAR + CHECK` | Padrão diferente do de cima | Tipos ENUM nativos (um padrão só) |

Mantidos de propósito (cache justificado, explicar na RFC):

- `account.balance` e `position.quantity` / `position.invested_amount`: são a linha travada com `FOR UPDATE`; o invariante com o ledger e as ordens é testado.
- `ledger_entry.balance_after`: usado no extrato e na resposta idempotente.
- `investment_order.cost_amount`: necessário no resgate (custo médio).

**Resultado:** 21 → 17 tabelas, cerca de 20 colunas a menos, um dono só para o dinheiro (a conta), um padrão só de enumerador e um lugar só para a chave de idempotência.

---

## Fases

Toda fase que mexe em `database/database.sql` exige `docker compose down -v` e `docker compose up -d --build`. TDD: teste black box primeiro (R1), ver falhar, implementar. A resposta da API não muda, salvo onde está dito.

### Fase 1: o pedido de dinheiro reorganizado (idempotência + donos)

- Testes em `tests/integration/test_transaction.py` e `test_investment_order.py`:
  - chave de um depósito reusada numa aplicação → 409 (hoje dá 500);
  - chave de uma ordem reusada na rota de transação → 409;
  - chave de outra conta / outro cliente → 409, sem dados do outro;
  - mesma chave com outro tipo, valor, destino ou produto → 409 (comparando com as colunas);
  - resgate parcial e depois `redeem_all` com a mesma chave → 409;
  - chave que não é UUID → 400.
- `database.sql`:
  - `transaction`: `account_id` + `destination_account_id`; `idempotency_key UUID UNIQUE` (aceita nulo); sem `request_hash`;
  - `investment_order`: só `order_key`, `transaction_id UNIQUE`, `product_id`, `quantity`, `unit_price`, `cost_amount`, `redeem_all`, `maturity_date`, `settled_order_id`, `created_at`;
  - `position`: `account_id` no lugar de `customer_id`.
- Controllers de transação e de ordem: sai `hash_request`; busca global da chave; dono ou rota diferente → `IdempotencyKeyReused`; validação de UUID; liquidação sem chave (sai o `uuid5`).
- Repositórios, modelos e DTOs de `transaction`, `investment_order` e `position` (filtrar pela conta, comprovante e contraparte pelo `account_id` / `destination_account_id`).
- Fechamento de conta: "tem posição aberta" passa a olhar a conta.

### Fase 2: conta do banco para a tarifa

- Testes: a transferência credita a tarifa na conta do banco; a conservação do dinheiro fecha.
- `database.sql`: seed da conta do banco (`account.customer_id` nulo só nela; o R8 nunca a alcança porque toda rota filtra por cliente).
- `transaction_controller.transfer`: 4 lançamentos (débito do principal, débito da tarifa, crédito do principal, crédito da tarifa no banco), travando a conta do banco junto, em ordem crescente de id.
- RFC: atualizar RN-REC-02/03 e a decisão "não existe conta de receitas" (R7: registrar a troca).

### Fase 3: enumeradores como ENUM nativo

- `database.sql`: `CREATE TYPE` para cada enumerador; apagar `account_status`, `transaction_type` e `product_type`; o índice parcial vira `WHERE status <> 'CLOSED'`.
- `transaction`: `CHECK ((type = 'TRANSFER') = (destination_account_id IS NOT NULL))`. Ficou para esta fase porque, com `type_id`, a regra dependeria de um id "mágico" do seed.
- Modelos: `sqlalchemy.Enum(..., name=..., create_type=False)` (o SQLAlchemy já está instalado).
- Controllers e DTOs: `x.status.enumerator` vira `x.status`; somem `TransactionRepository.get_type` e afins.
- Seeds de produto e carteira deixam de fazer join com tabela de tipo.

### Fase 4: colunas redundantes restantes

Uma subfase por módulo, na ordem do projeto:

1. movimentações (`entry_key`);
2. suitability (`position`, `points`, `answer.question_id`, `valid_until`, `forced_by_rule`);
3. produtos (`own_issue`; atualizar `docs/precificacao-e-fechamento-do-dia.md`);
4. carteira (`model_portfolio` some).

Os campos que saem na API (`forced_by_rule`, `valid_until`) continuam, calculados no DTO ou no controller; os testes existentes provam que nada mudou para fora.

### Fase 5: capital total do cliente na rota de posições

- Testes em `tests/integration/test_investment_order.py`: cliente só com caixa (100% caixa); depois de aplicar, `net_worth_amount = cash_amount + current_amount` e `cash_bps + investments_bps = 10000`; cliente sem nada (tudo 0).

- `investment_order_controller.positions`: passa o saldo da conta ao DTO.
- `investment_order_dto.total_to_dict`: soma e percentuais. Nenhuma mudança no banco.
- RFC: RN-POS (novos campos) e a decisão "calcular na leitura × guardar coluna" (R7).

### Fase 6: documentação sem divergência

- `docs/diagramas/er-*.mmd`: **todas** as colunas de cada tabela, iguais ao `database.sql`. Hoje faltam, por exemplo, `transaction.description`, `suitability_question.text/position`, `product.name/issuer/indexer/rate_description/own_issue`, `investment_order.status` e vários `created_at`. Regerar os PNG com `npx -p @mermaid-js/mermaid-cli mmdc`.
- Script pequeno `scripts/check_er.py` (só biblioteca padrão): lê os `CREATE TABLE` do SQL e os blocos dos `.mmd` e lista as diferenças. Roda à mão depois de mexer no SQL.
- `docs/regras_de_negocio.tex`: tabela de colunas, RN-GER-08, RN-ORD-15, RN-FEC-09, catálogo `PPP000004` ("chave já usada por outro pedido") e decisões descartadas (R7: hash, chave por dono, donos misturados, tabelas de enumerador, receita sem conta). Apagar o diagrama mermaid duplicado do apêndice e apontar para os `.mmd`.
- `docs/rfc_fluxos.tex` e `docs/bootcamp-rfc-v0.md`: passos de idempotência e de transferência.

---

## Verificação

- Fim de cada fase: `docker compose down -v`, `docker compose up -d --build`, esperar a API ficar healthy e rodar `pytest -v` (tudo verde).
- Fase 1: o teste "chave do depósito reusada na aplicação" falha com 500 antes e passa depois.
- Fase 2: soma dos saldos (com o banco) = depósitos − saques − aplicações + resgates.
- Fase 5: `GET /positions` de um cliente com caixa e aplicação mostra o total e os percentuais.
- Fase 6: `python scripts/check_er.py` sem diferenças; PNGs abertos e conferidos.
- `grep -rn "request_hash\|requester_account_id\|origin_account_id\|own_issue\|status_id" src docs` sem sobras.